DevOps Lab 5

Build, Test, and Package Applications with IntelliJ IDEA and Docker

Name	Sadia
Roll No.	2023-BCS-075
Course / Section	DevOps (DevOps_F26) / Section A
GitHub Username	sadia612
Repository	https://github.com/Sadia612/DevOps
Submission Directory	https://github.com/Sadia612/DevOps/tree/main/Labs/Lab05
Environment	Ubuntu 26.04 LTS (VMware), Docker, IntelliJ IDEA 2026.2

Contents

Task 1	Clone and Launch IntelliJ IDEA	3 screenshots
Task 2	Build and Run the Maven Project	10 screenshots
Task 3	Build and Run the Gradle Project	8 screenshots
Task 4	Install and Run the Node.js Project	7 screenshots
Task 5	Build the React and Node.js API Artifact	4 screenshots
Task 6	Build a Docker Image for the Node.js Project	5 screenshots
	Exam Evaluation Questions (practice answers)	

Task 1: Clone and Launch IntelliJ IDEA

The supplied IntelliJ Docker repository was cloned on the Ubuntu host. Its setup script built the image and started the container with Docker Compose. IntelliJ IDEA was then opened in the browser desktop on port 3001.

Commands used

```
git clone https://github.com/WaqasSaleem97/Intellij-Idea-docker.git
cd Intellij-Idea-docker
bash setup.sh
docker compose ps
```

Screenshots



Figure 1: The IntelliJ Docker repository cloned successfully on the Ubuntu host. (intellij_repo_clone.png)

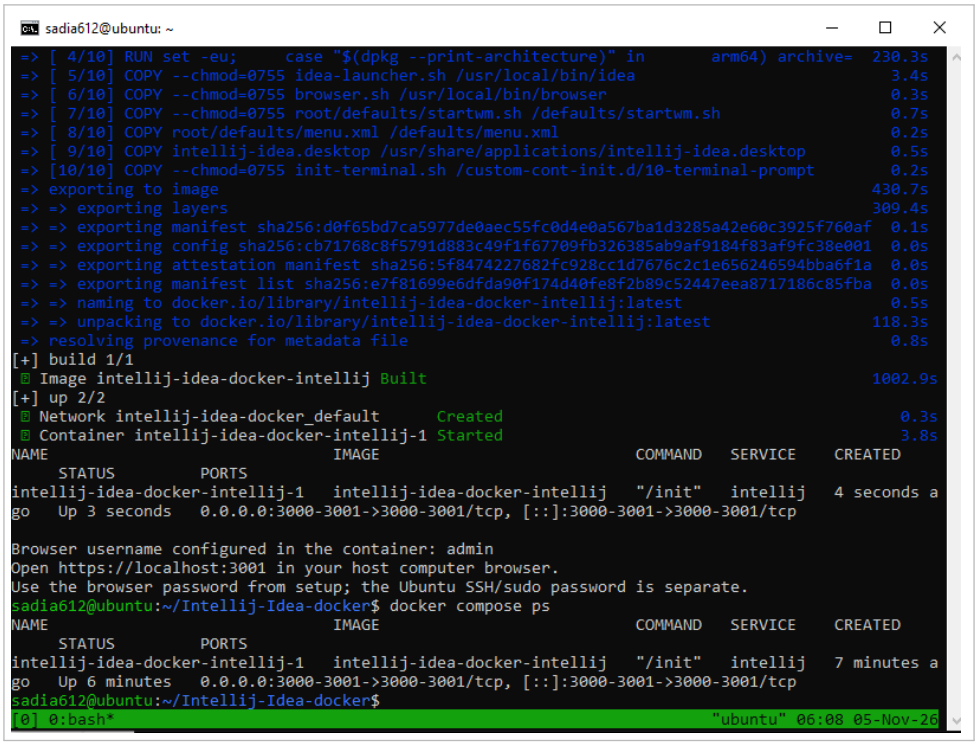


Figure 2: The intellij service is running and the desktop port 3001 is published. (intellij_running.png)

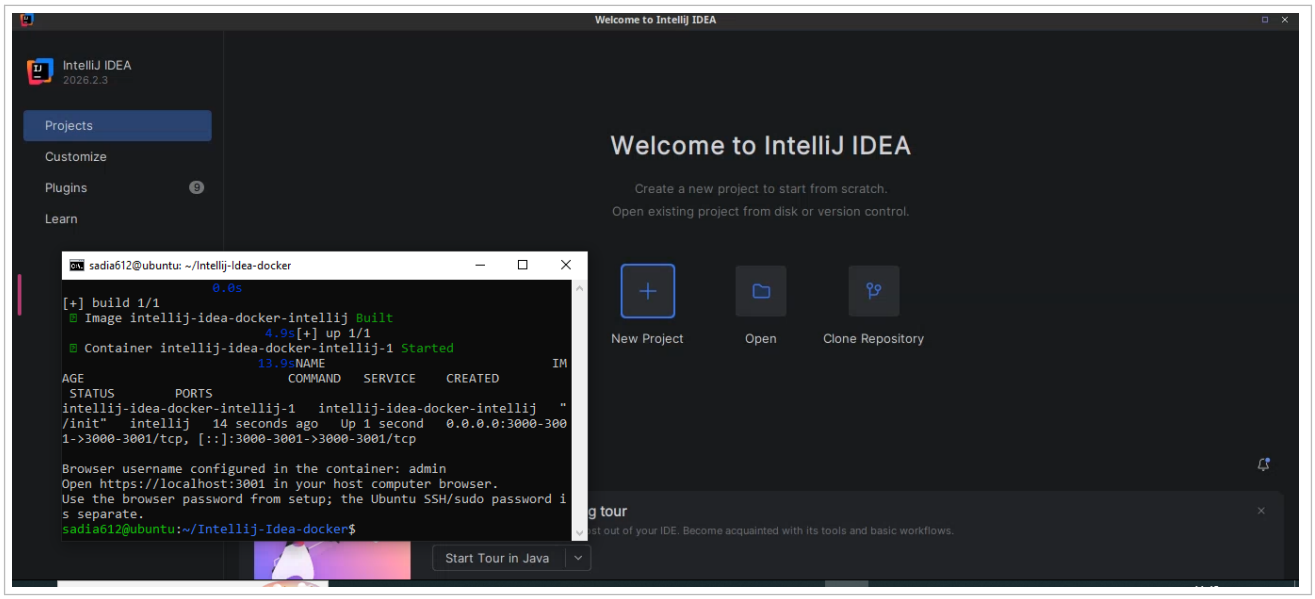


Figure 3: IntelliJ IDEA Welcome screen in the browser desktop, with the host terminal beside it. (intellij_desktop.png)

Task 2: Build and Run the Maven Project

The java-maven-app project was cloned in IntelliJ. Maven was installed, the project was packaged, the Elasticsearch 7.0.0 dependency was added to pom.xml and synchronized, then the project was packaged again, tested and the generated JAR was run.

Commands used

```
sudo apt update
sudo apt install -y maven
mvn --version
mvn package
mvn package        # after adding the Elasticsearch dependency
mvn test
java -jar target/java-maven-app-1.1.0-SNAPSHOT.jar
```

Screenshots

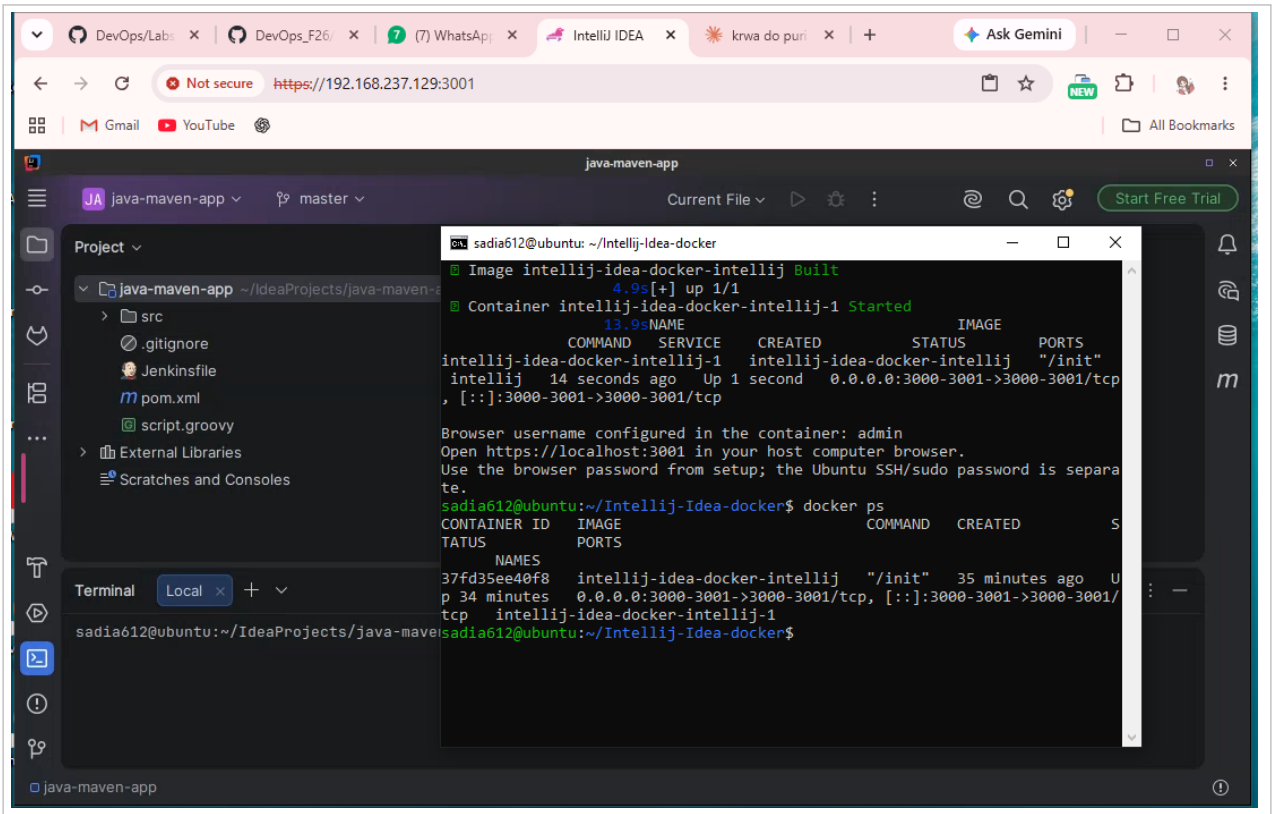


Figure 4: Clone dialog with the java-maven-app URL and destination directory. (maven_project_clone.png)

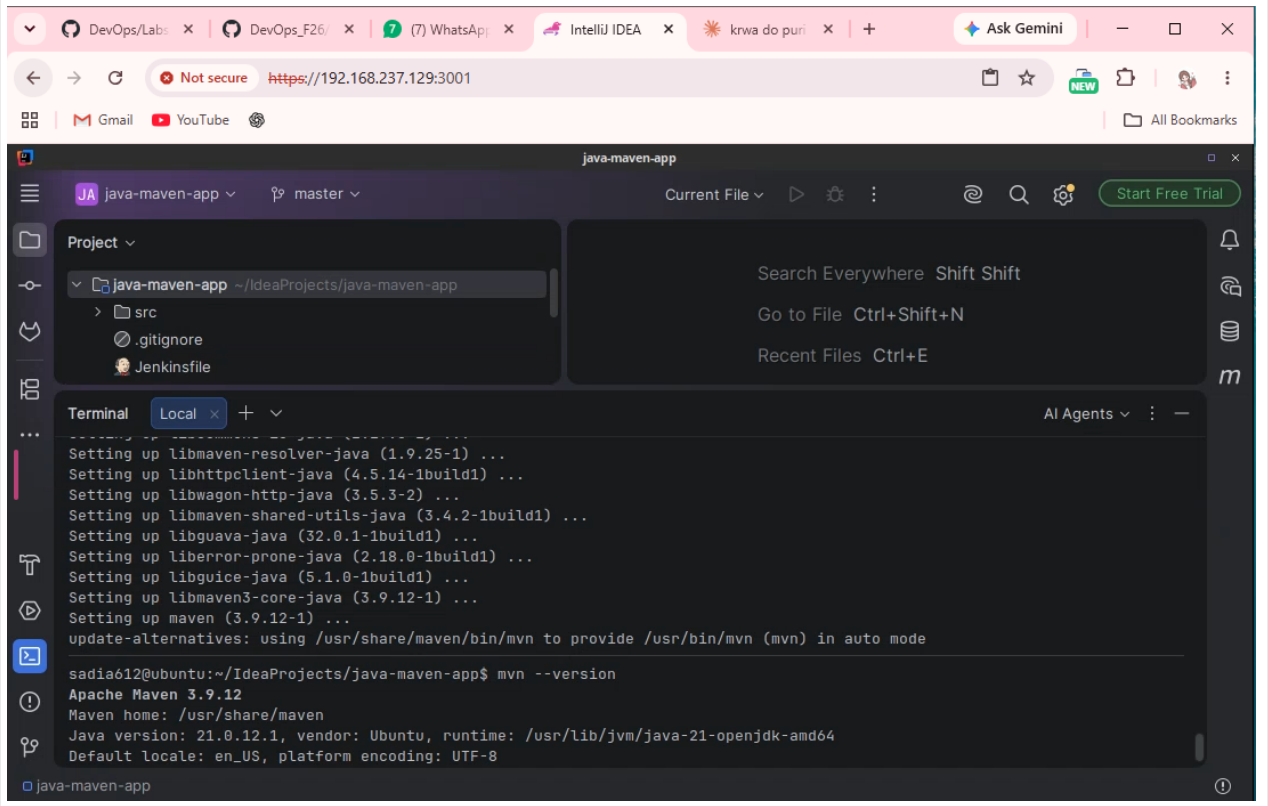


Figure 5: Maven installed; mvn --version printed in the IntelliJ terminal. (maven_installed.png)

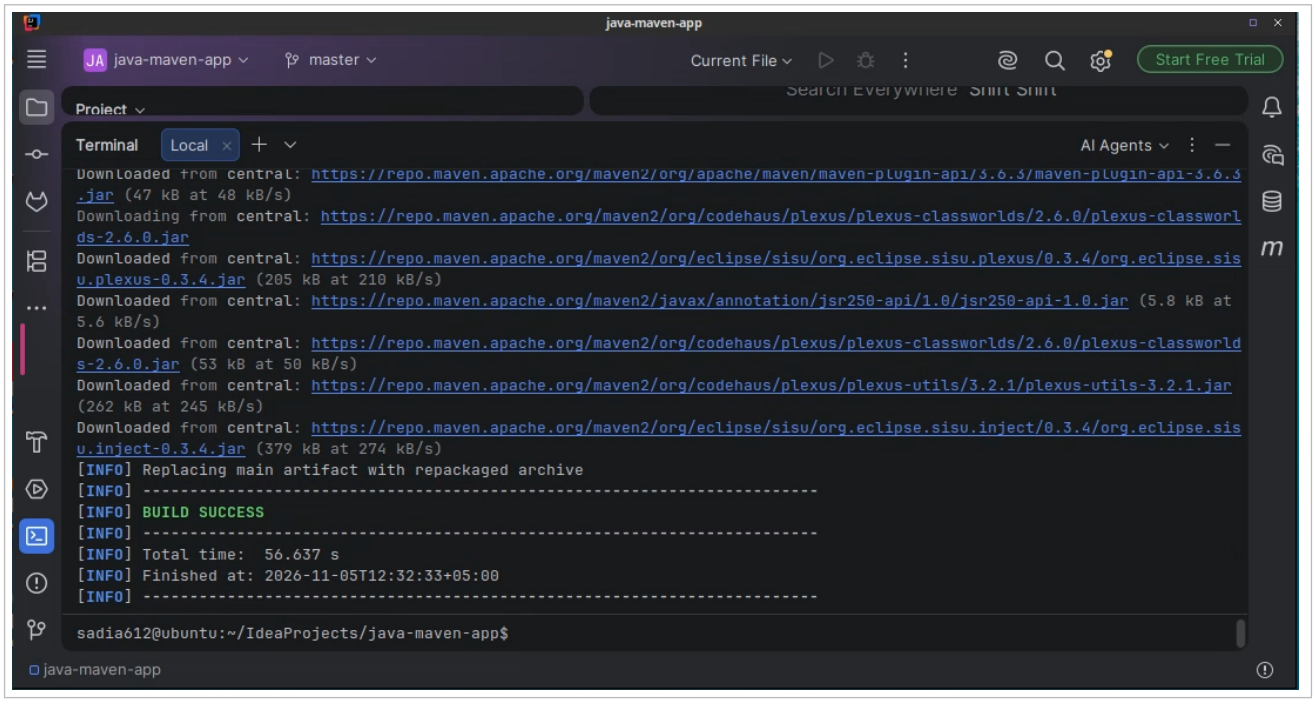


Figure 6: mvn package finished with BUILD SUCCESS. (maven_package.png)

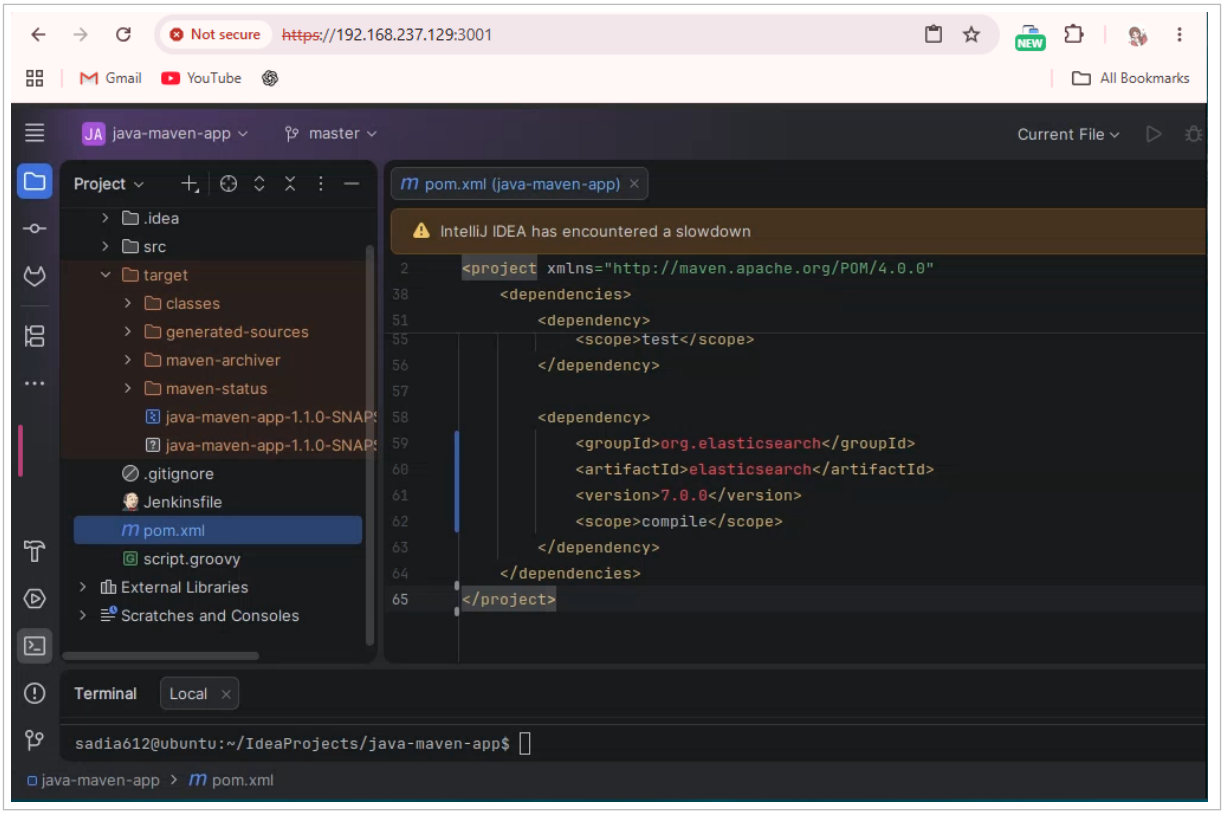


Figure 7: Elasticsearch 7.0.0 added to pom.xml and shown in the synchronized Maven dependencies. (maven_elasticsearch_dependency.png)

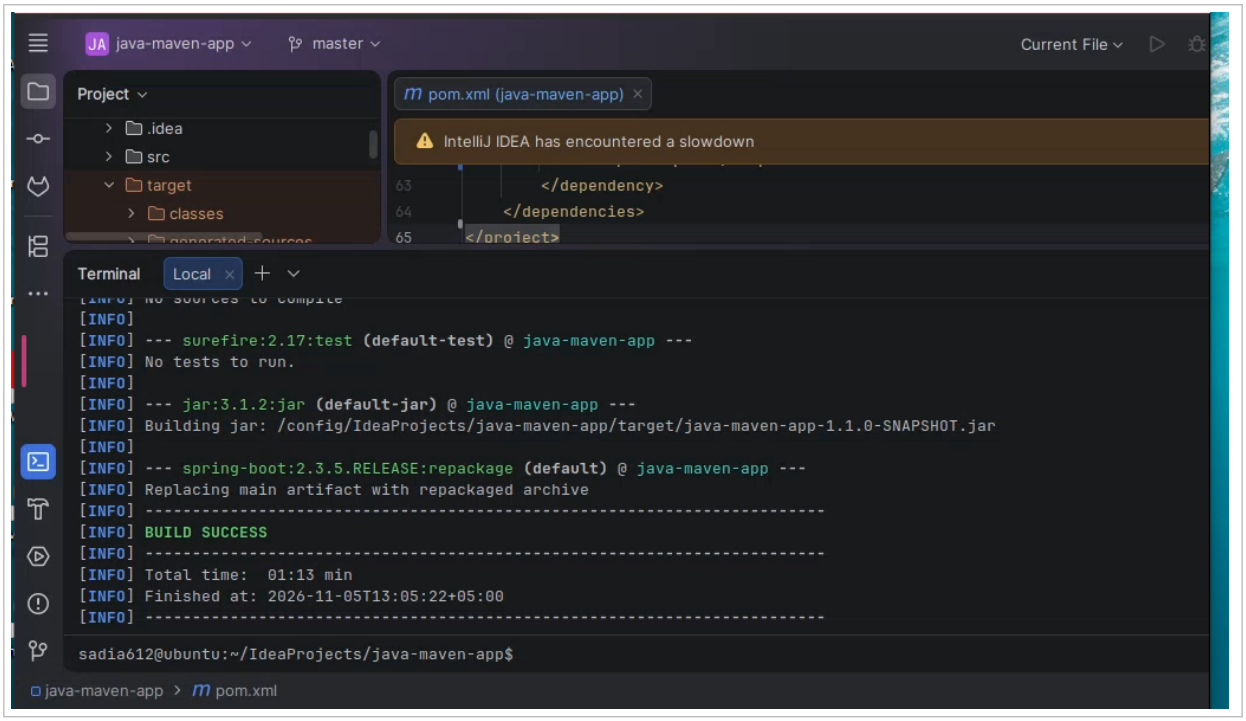


Figure 8: mvn package after adding the dependency finished with BUILD SUCCESS. (maven_repackage.png)

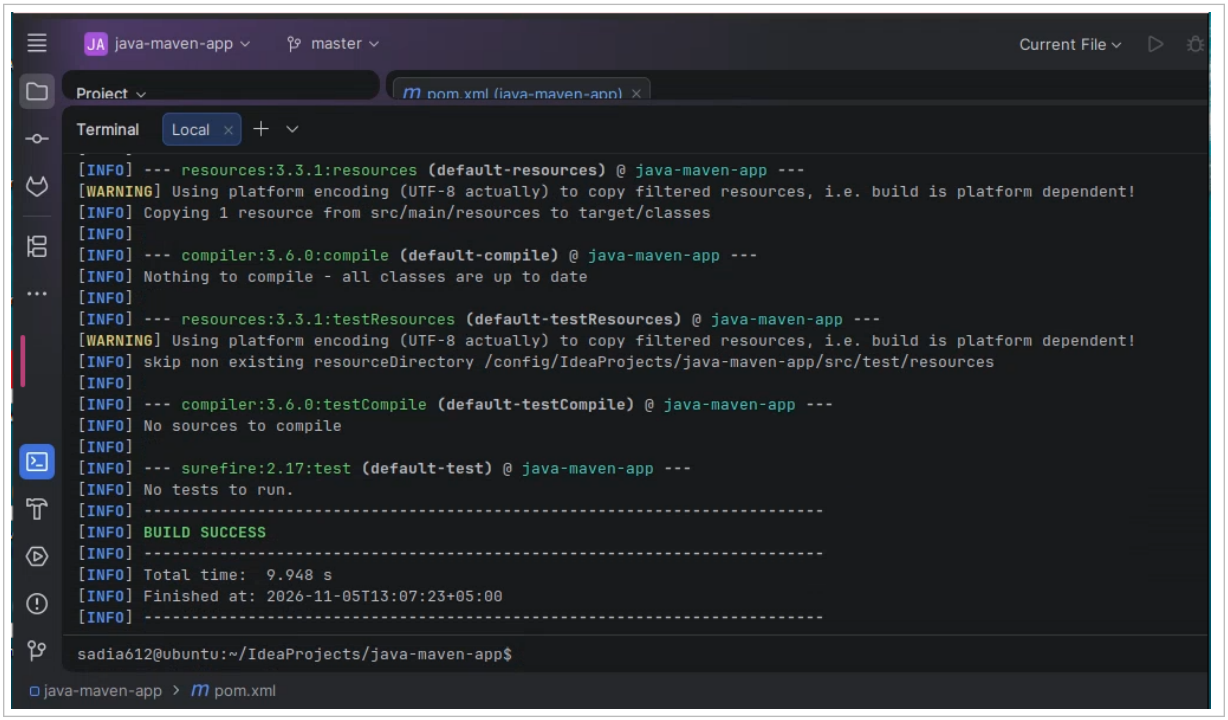


Figure 9: mvn test: the test phase ran (no test sources in this sample) with BUILD SUCCESS. (maven_test.png)

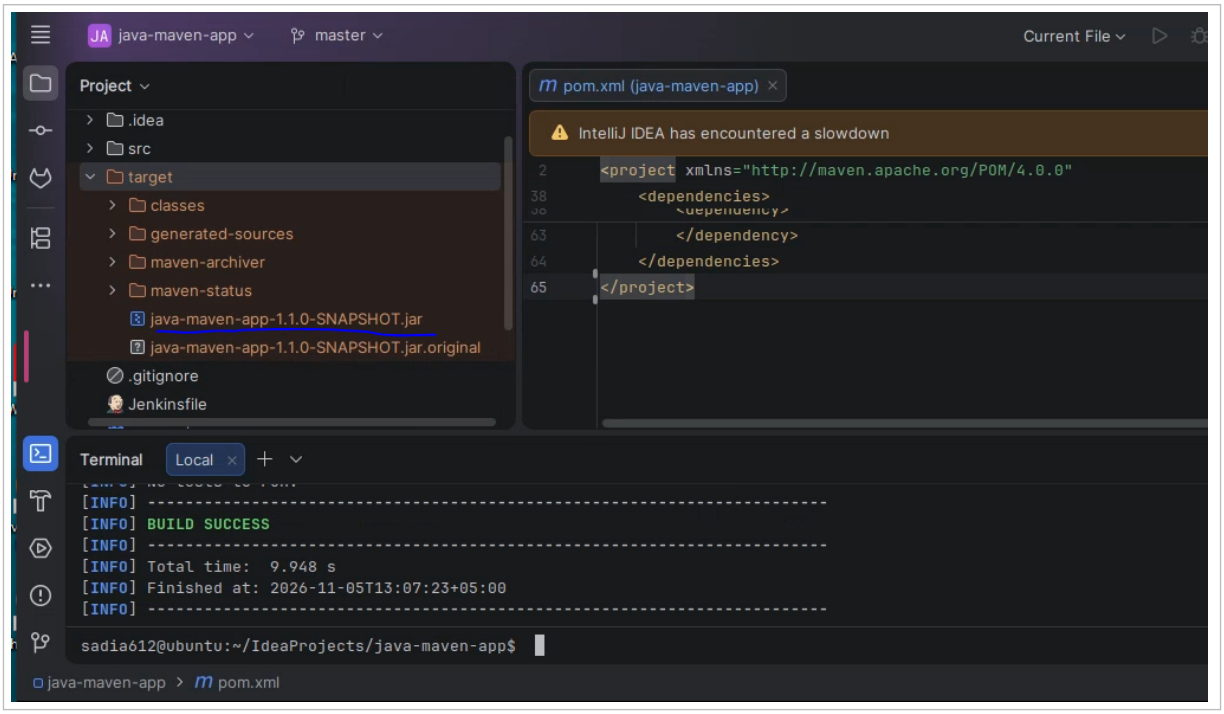


Figure 10: The generated target/java-maven-app-1.1.0-SNAPSHOT.jar in the project pane. (maven_jar.png)

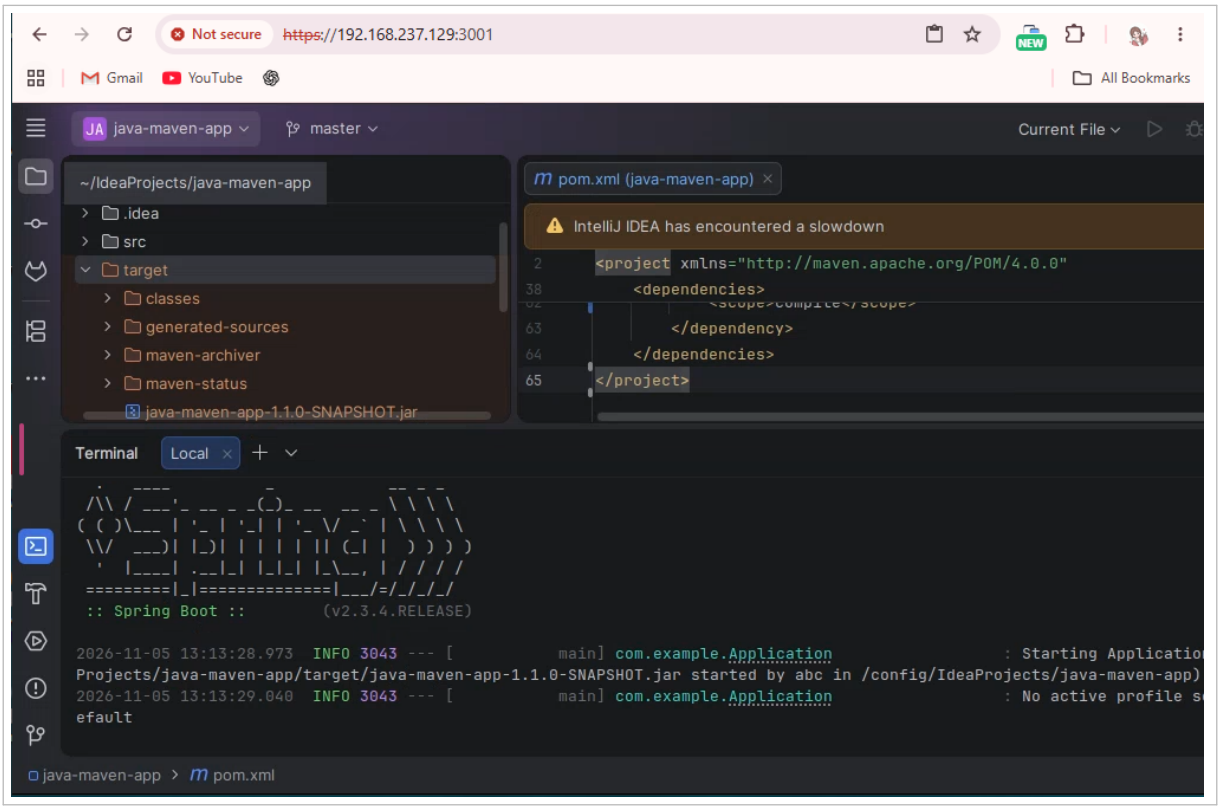


Figure 11: The JAR started as a Spring Boot web application. (maven_running.png)

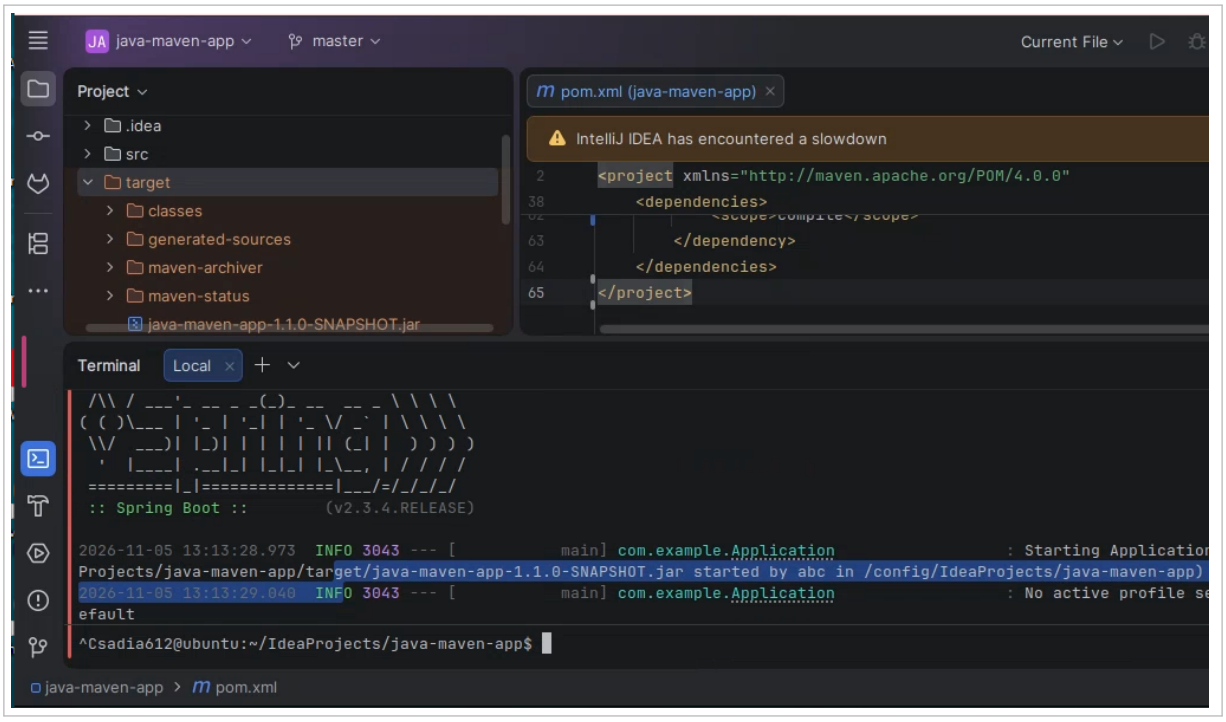


Figure 12: Application stopped with Ctrl+C and the shell prompt returned. (maven_stopped.png)

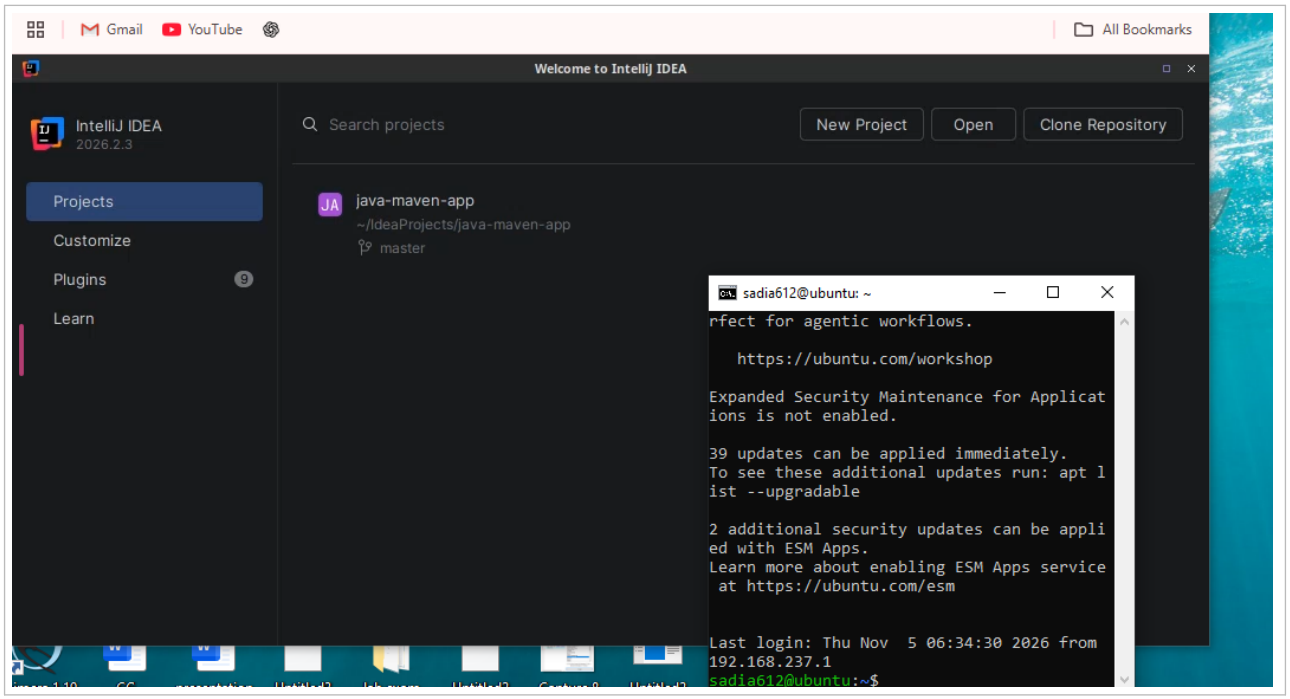


Figure 13: Project closed; IntelliJ Welcome screen shown. (maven_project_closed.png)

Task 3: Build and Run the Gradle Project

The java-app project was cloned and opened as a Gradle project. The Project SDK and the Gradle JVM were set to JDK 16 so that the Gradle Wrapper (7.x) could run. The project was built, the Elasticsearch 7.0.0 dependency was added to build.gradle and synchronized, and the project was rebuilt. The generated JAR was then run.

Commands used

```
./gradlew --version
./gradlew build
./gradlew build    # after adding the Elasticsearch dependency
java -jar build/libs/java-app-1.0-SNAPSHOT.jar
```

Screenshots

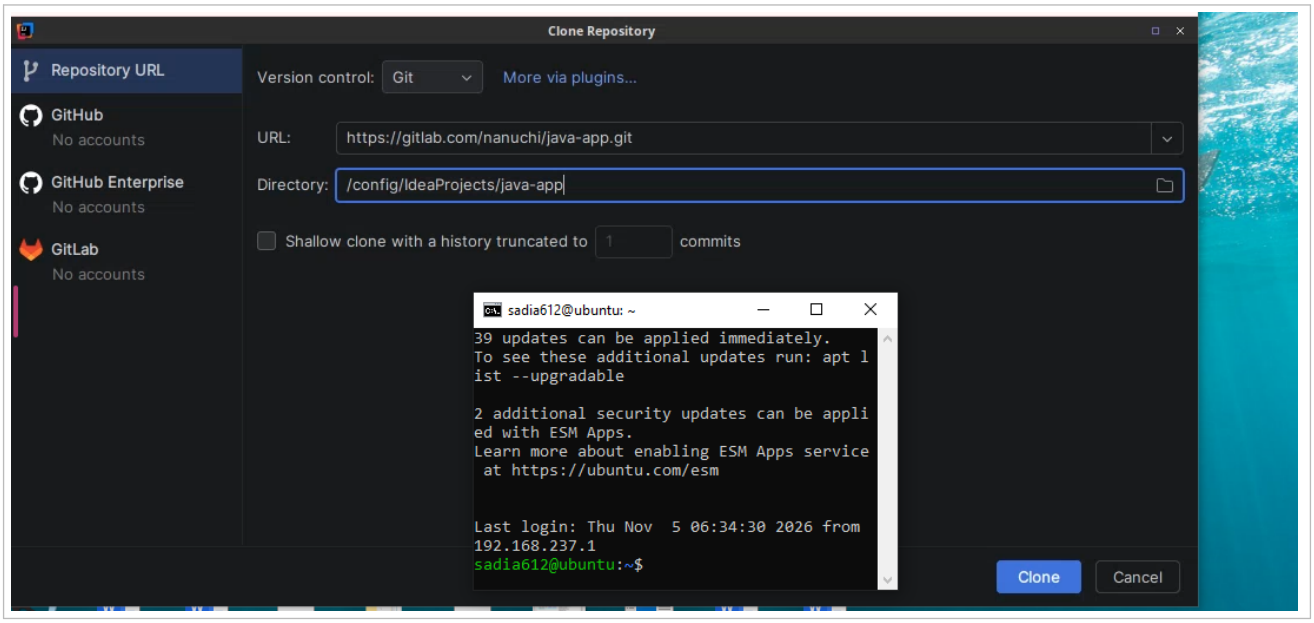


Figure 14: Clone dialog with the java-app URL and destination directory. (gradle_project_clone.png)

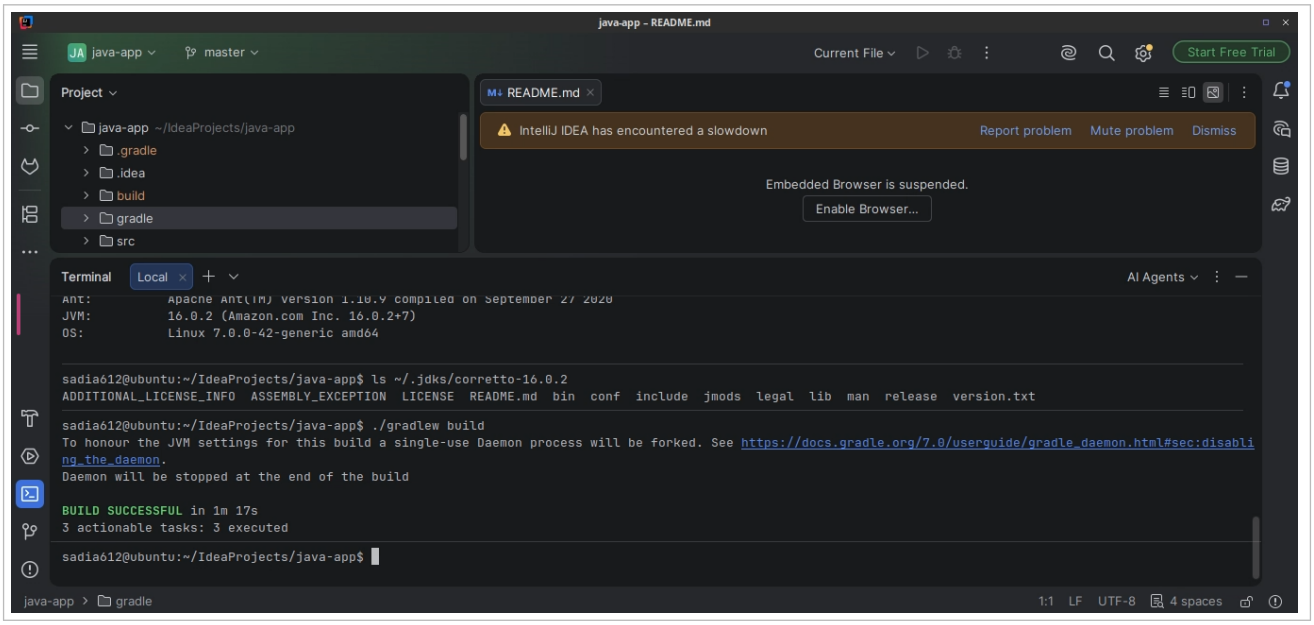


Figure 15: ./gradlew build finished with BUILD SUCCESSFUL. (gradle_build.png)

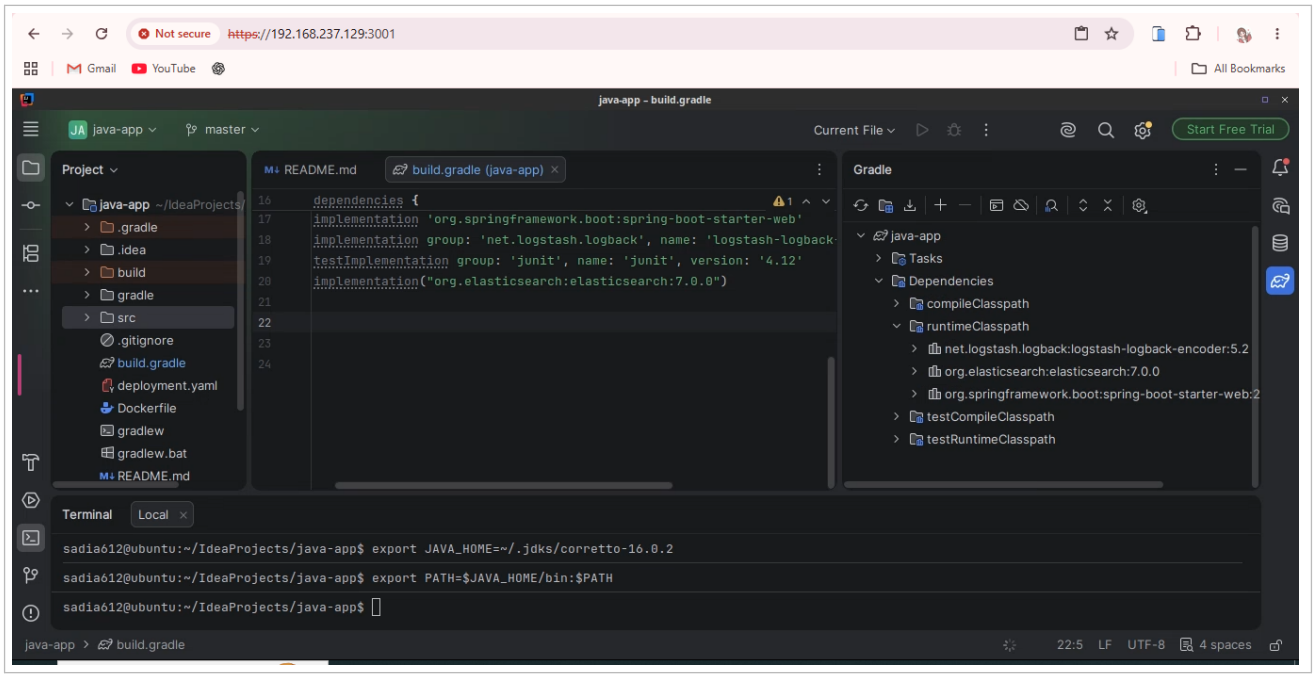


Figure 16: Elasticsearch 7.0.0 added to build.gradle and visible in External Libraries after sync. (gradle_elasticsearch_dependency.png)

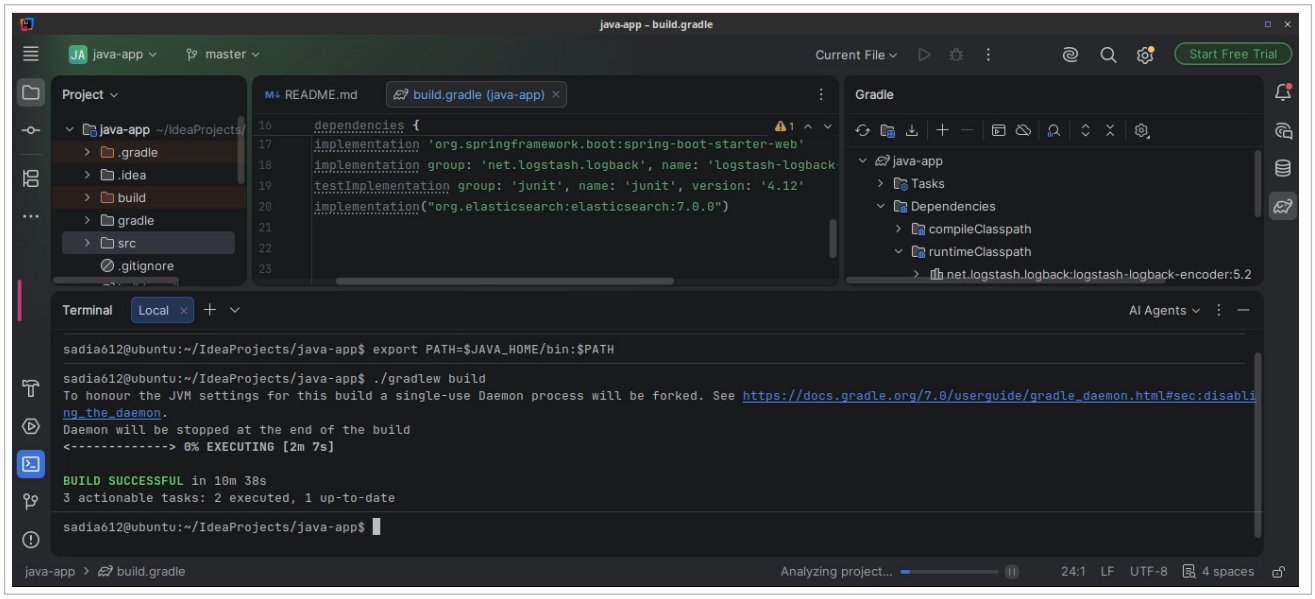


Figure 17: Rebuild after the dependency change finished with BUILD SUCCESSFUL. (gradle_rebuild.png)

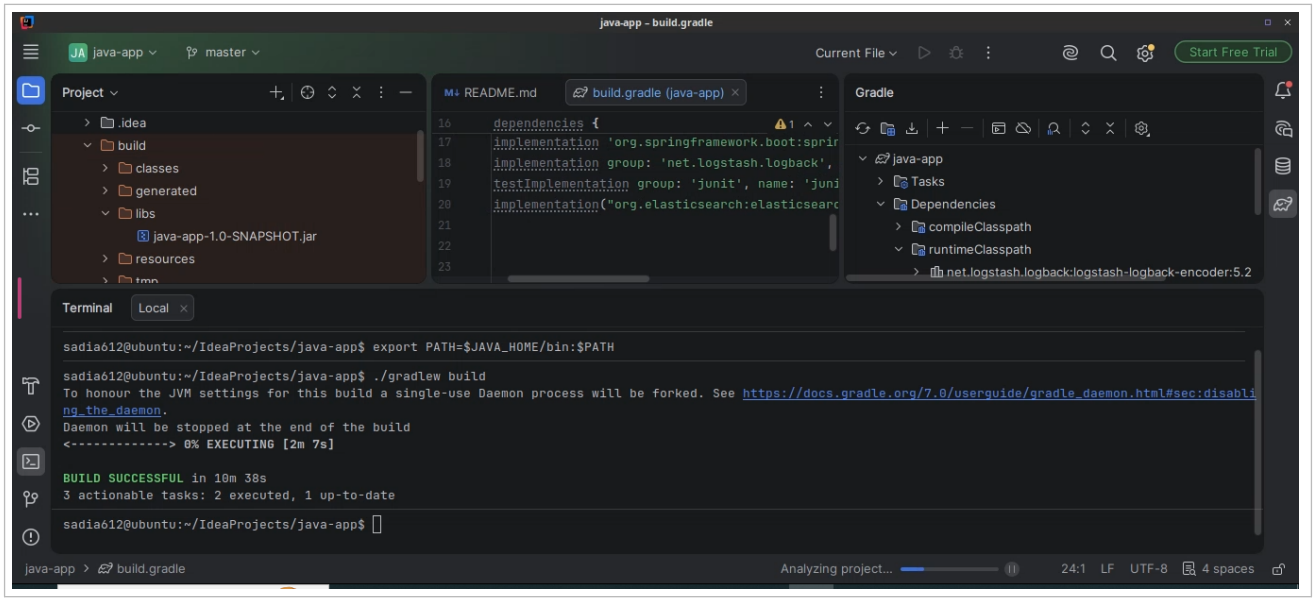


Figure 18: The generated build/libs/java-app-1.0-SNAPSHOT.jar in the project pane. (gradle_jar.png)

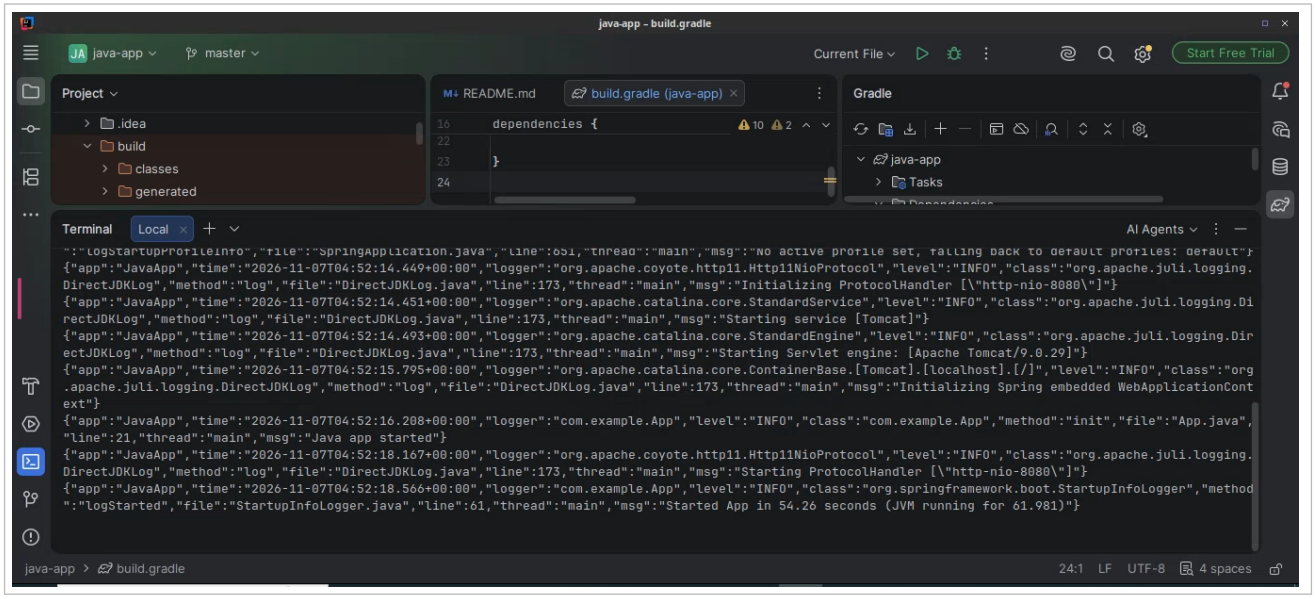


Figure 19: The JAR started successfully with java -jar. (gradle_running.png)

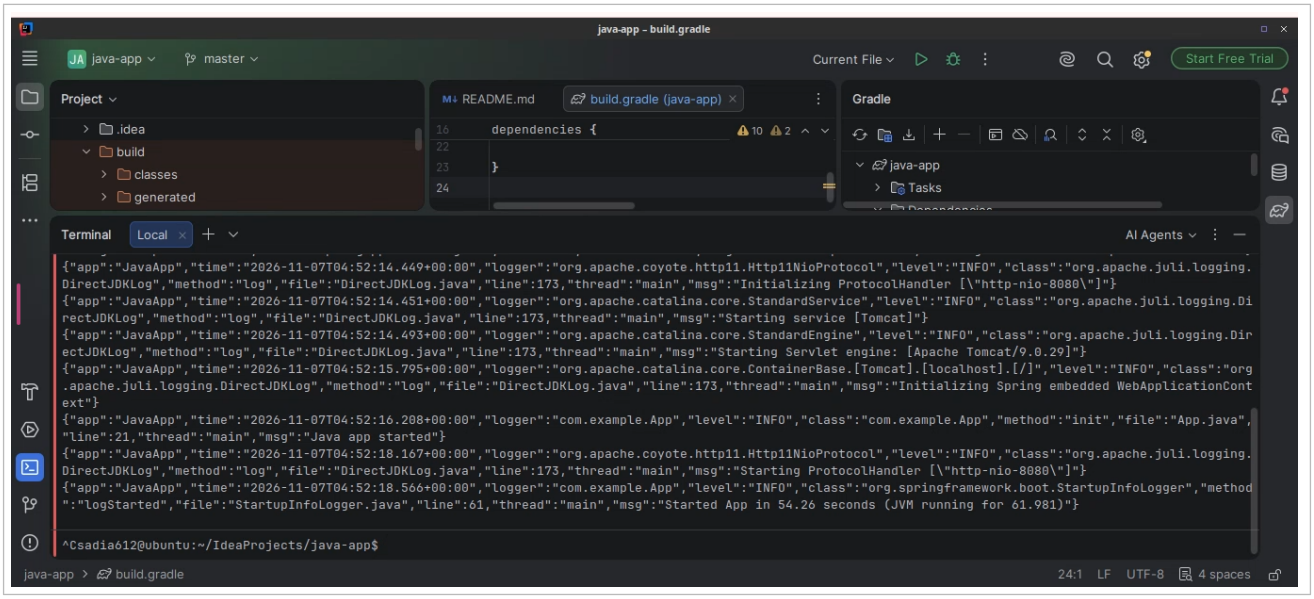


Figure 20: Application stopped with Ctrl+C and the shell prompt returned. (gradle_stopped.png)

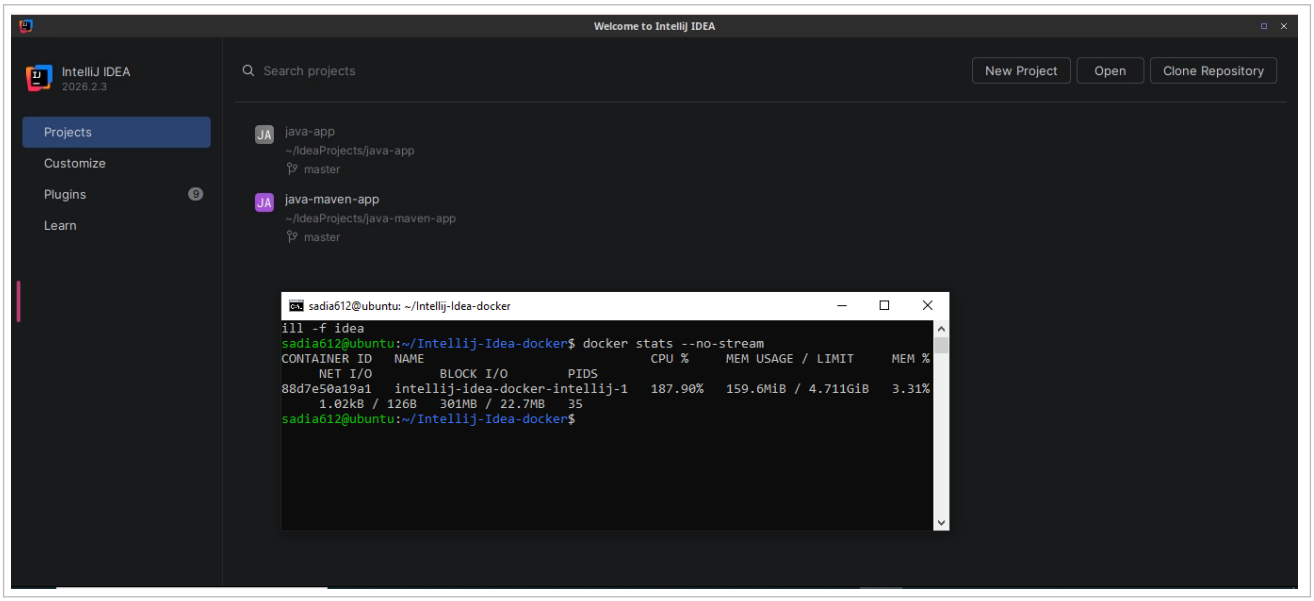


Figure 21: Project closed; IntelliJ Welcome screen shown. (gradle_project_closed.png)

Task 4: Install and Run the Node.js Project

The node-project repository was cloned. Node.js and npm were installed, the dependencies in the app directory were installed, and the application was started. Port 3000 is already used by the desktop inside this container, so npm start first failed with EADDRINUSE. The listener in app/server.js was changed to port 4000 and the application was opened in the container browser. Finally the Jest test was run and the package was created with npm pack.

Commands used

```
sudo apt update
sudo apt install -y nodejs npm
node --version
npm --version
cd app
npm install
npm start
npm test
npm pack
```

Screenshots

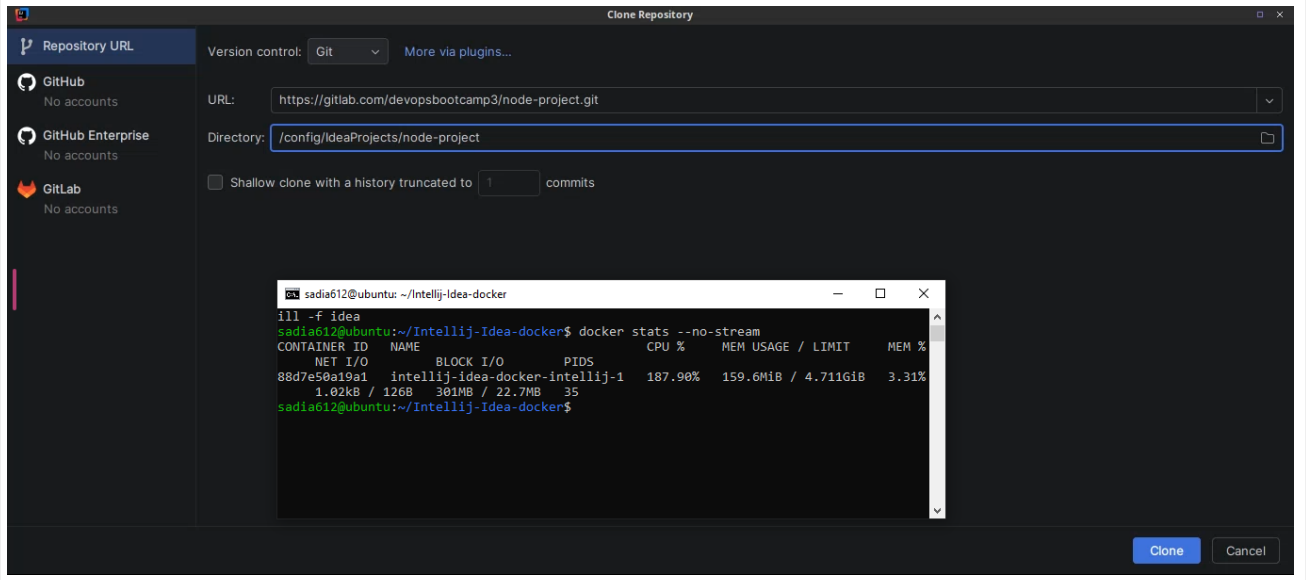


Figure 22: Clone dialog with the node-project URL and destination directory. (node_project_clone.png)

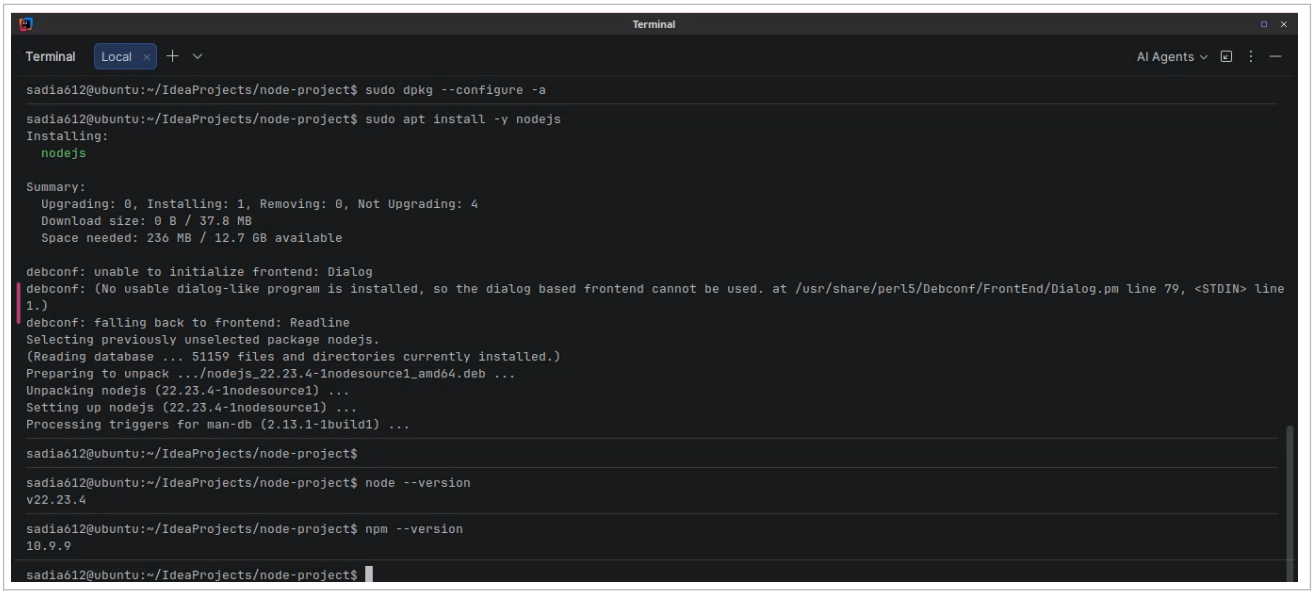


Figure 23: Node.js and npm installed; both versions printed. (nodejs_installed.png)

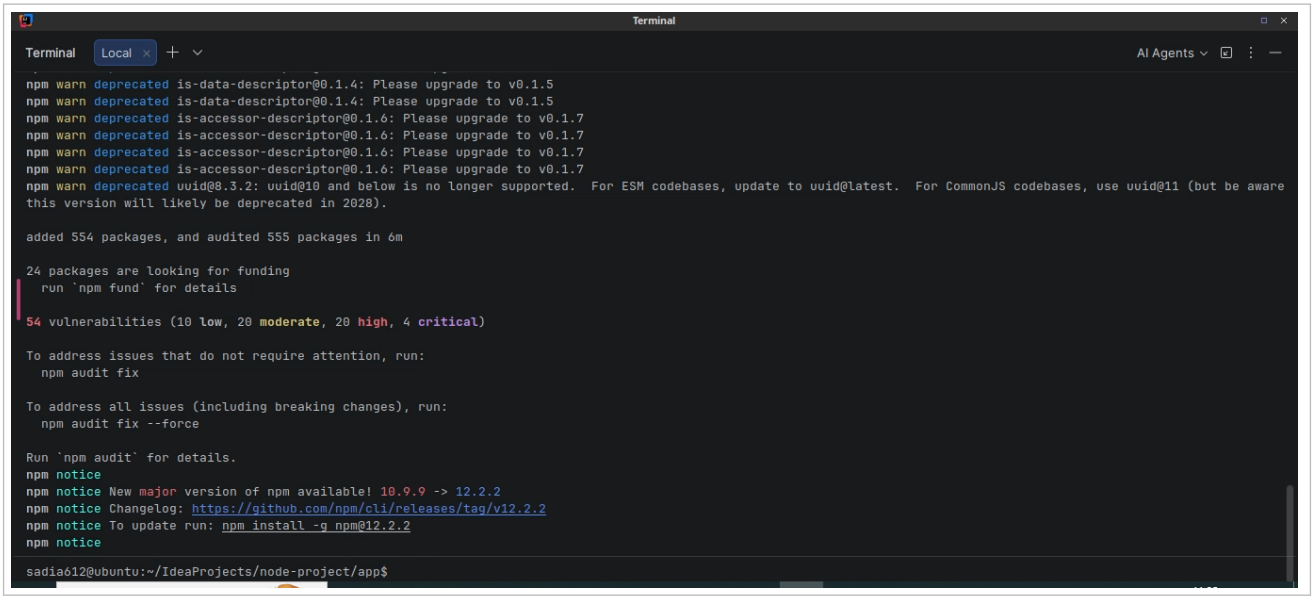


Figure 24: npm install completed in the app directory. (npm_install.png)

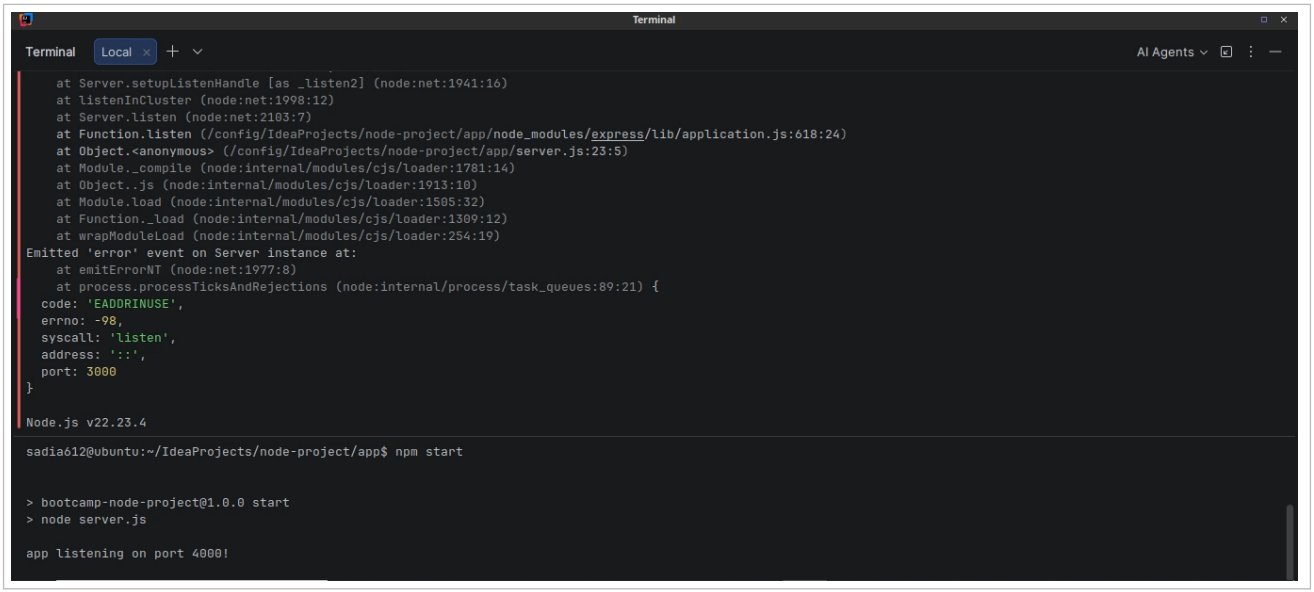


Figure 25: npm start with the application listening on port 4000. (node_running.png)

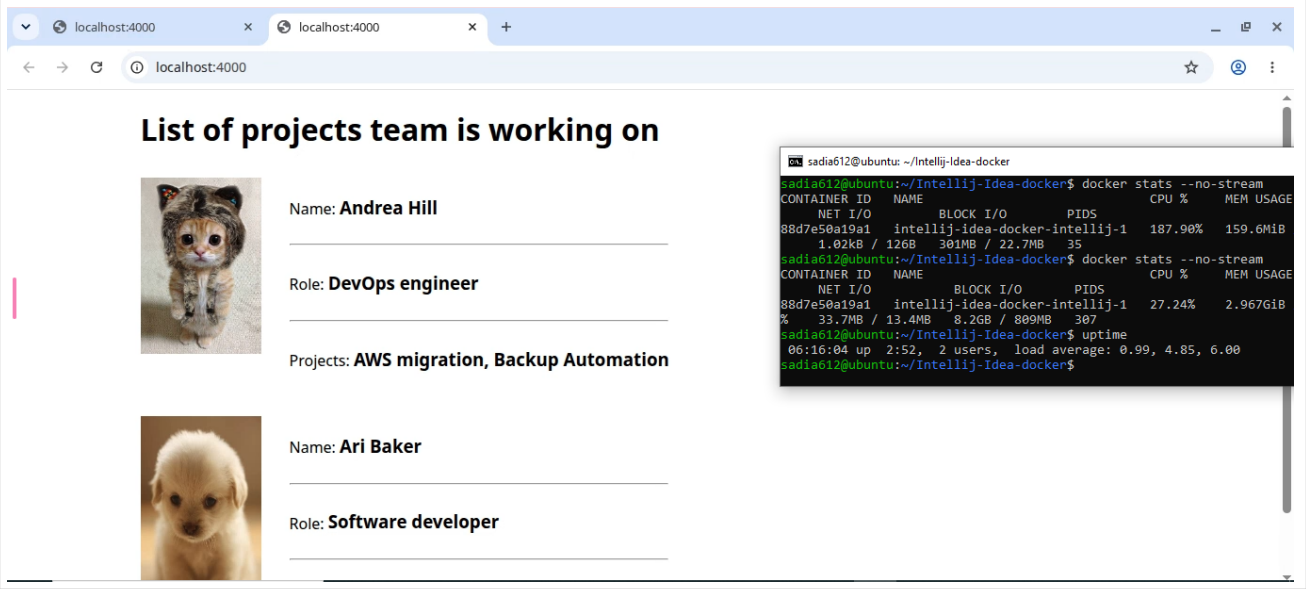


Figure 26: The Node.js project page open at http://localhost:4000 in the container browser. (node_browser.png)

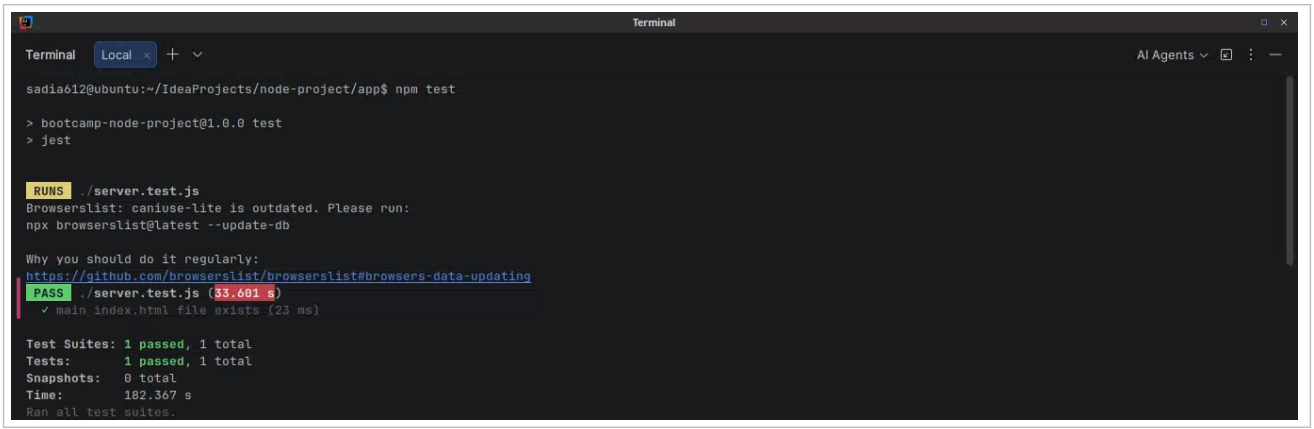


Figure 27: npm test: the Jest test passed. (npm_test.png)

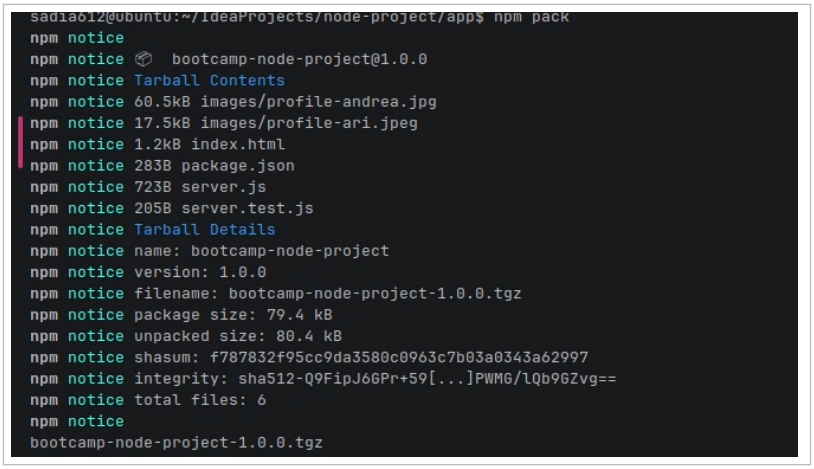


Figure 28: npm pack created bootcamp-node-project-1.0.0.tgz. (npm_pack.png)

Task 5: Build the React and Node.js API Artifact

The react-nodejs-example repository was cloned. In its api directory the dependencies were installed and webpack was used to create the bundled artifact api/server.bundle.js, which was then viewed with more.

Commands used

```
cd api
npm install
npm run build
cat server.bundle.js | more
```

Screenshots

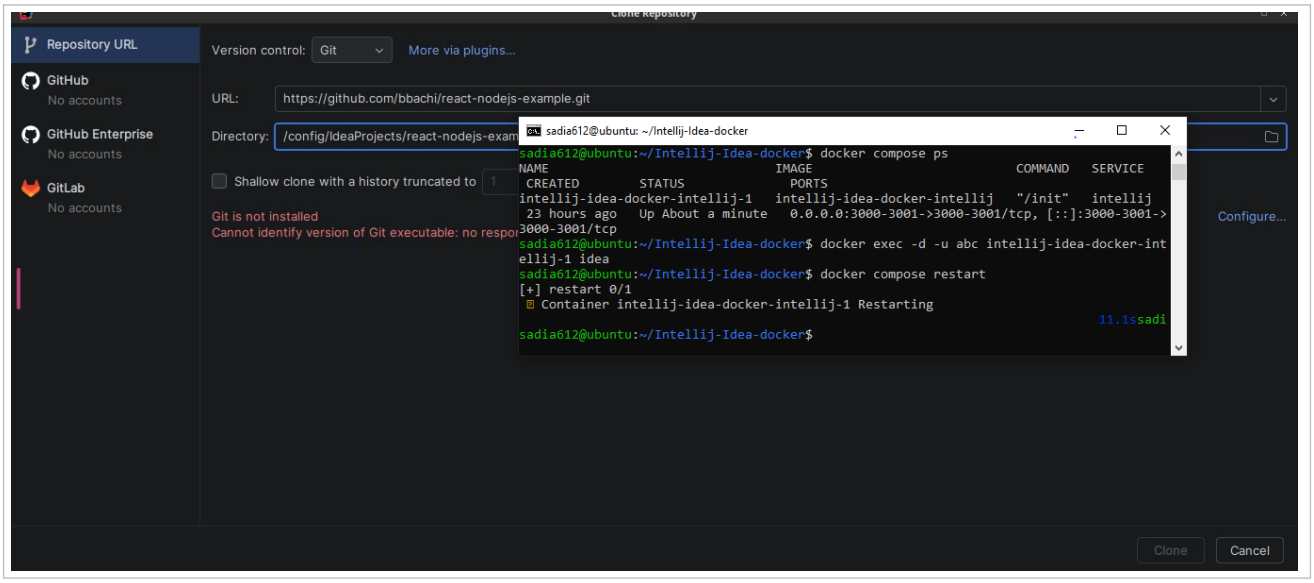


Figure 29: Clone dialog with the react-nodejs-example URL and destination directory. (react_node_project_clone.png)

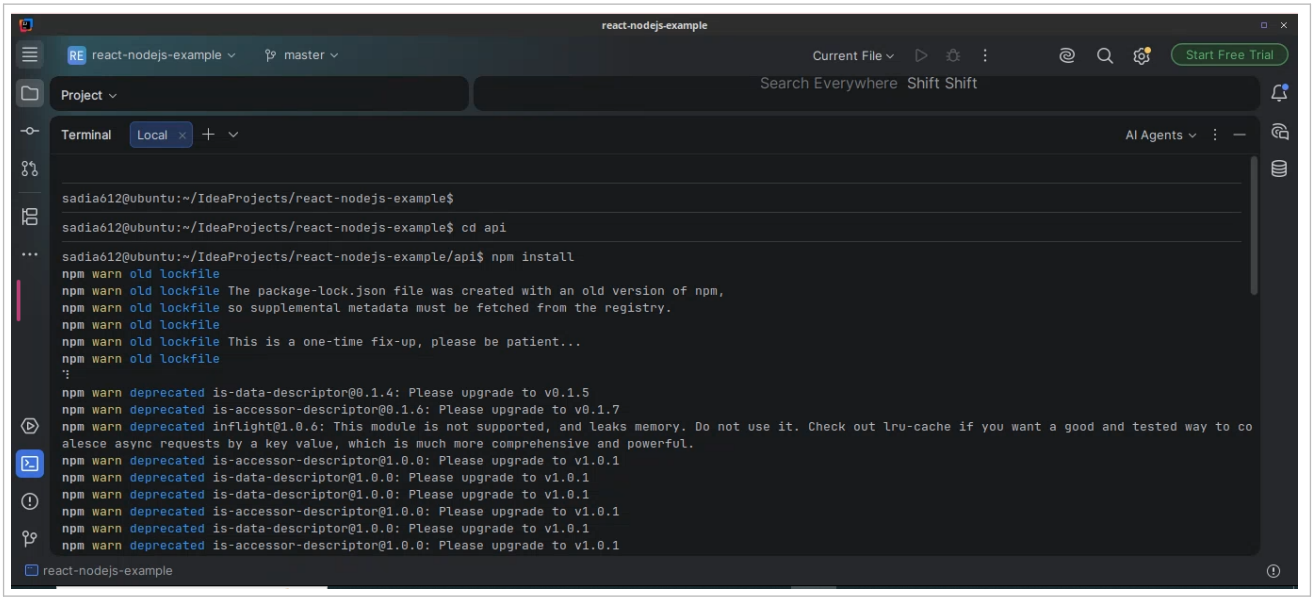


Figure 30: npm install completed in api and the node_modules directory is present. (react_api_dependencies.png)

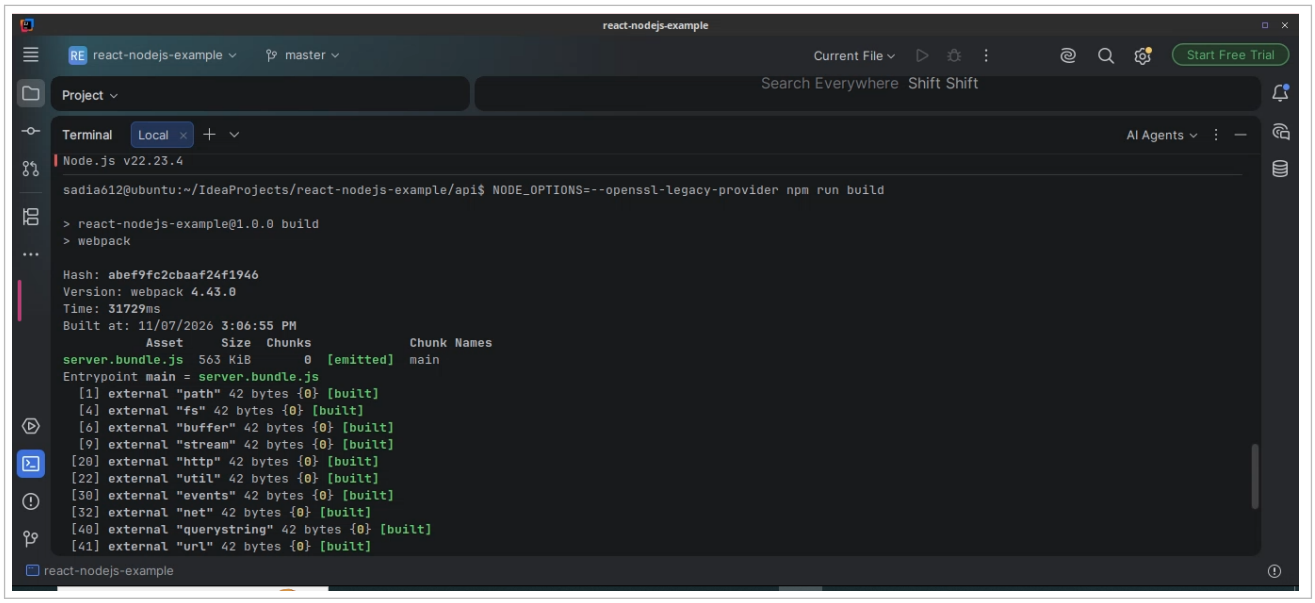


Figure 31: npm run build completed and server.bundle.js was generated in api. (react_api_build.png)

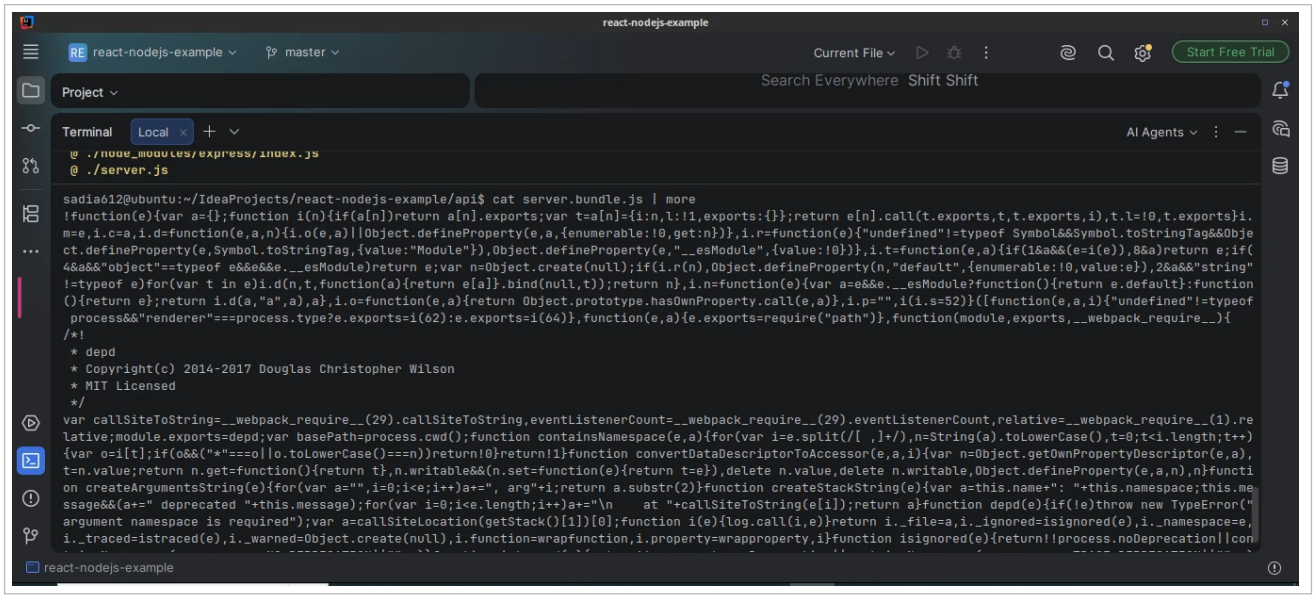


Figure 32: The bundled JavaScript viewed with cat server.bundle.js | more. (react_api_bundle.png)

Task 6: Build a Docker Image for the Node.js Project

From the root of the node-project clone, the existing Dockerfile was used to build the image lab5:1.0. The image list was checked before and after the build.

Commands used

```
cd /config/IdeaProjects/node-project
docker --version
ls
sudo docker images
sudo docker build -t lab5:1.0 .
sudo docker images
```

Screenshots

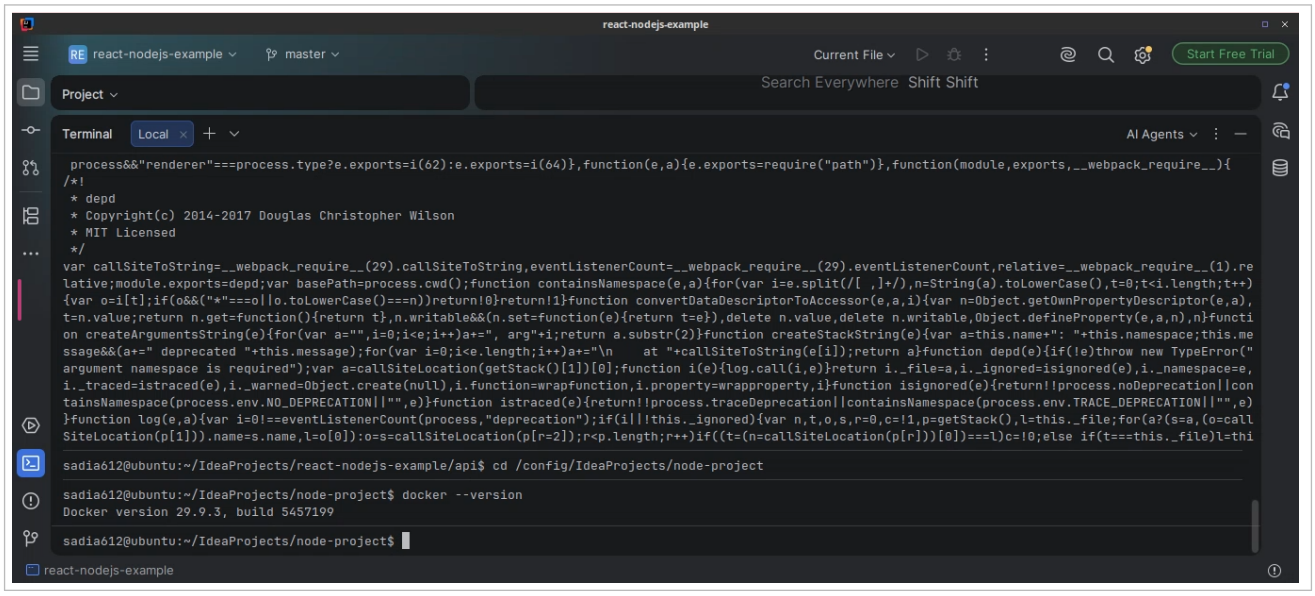


Figure 33: Docker is installed; docker --version printed. (docker_installed.png)

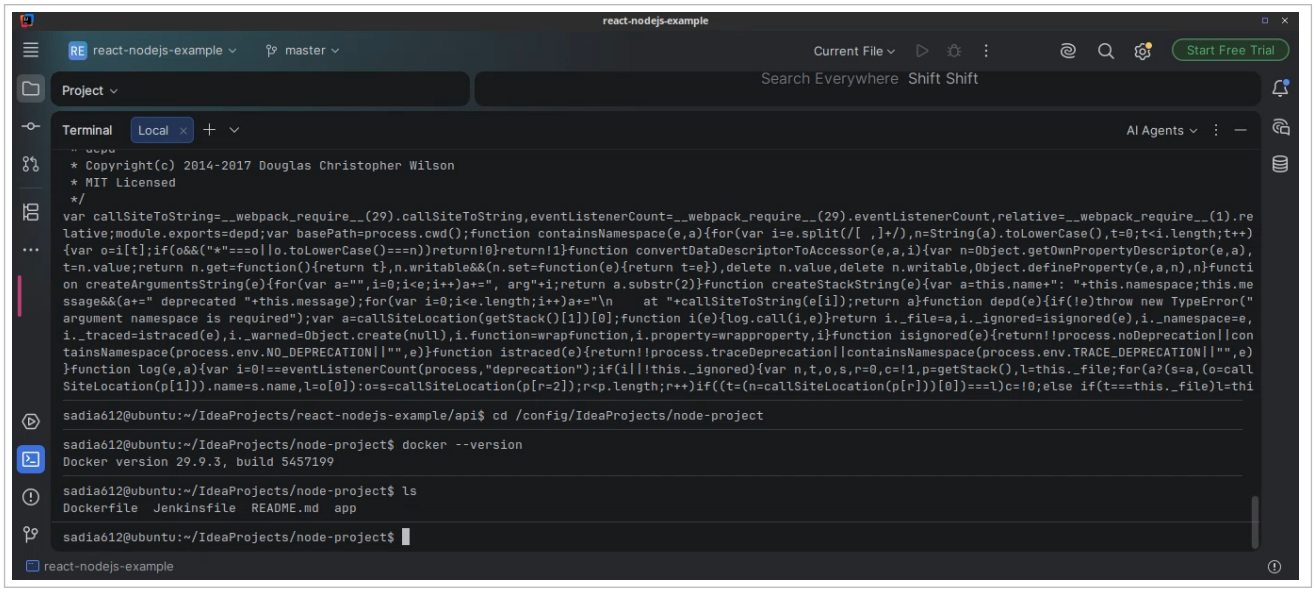


Figure 34: The Dockerfile is present in the repository root. (dockerfile_exists.png)



Figure 35: Image list before the build. (docker_images_before.png)

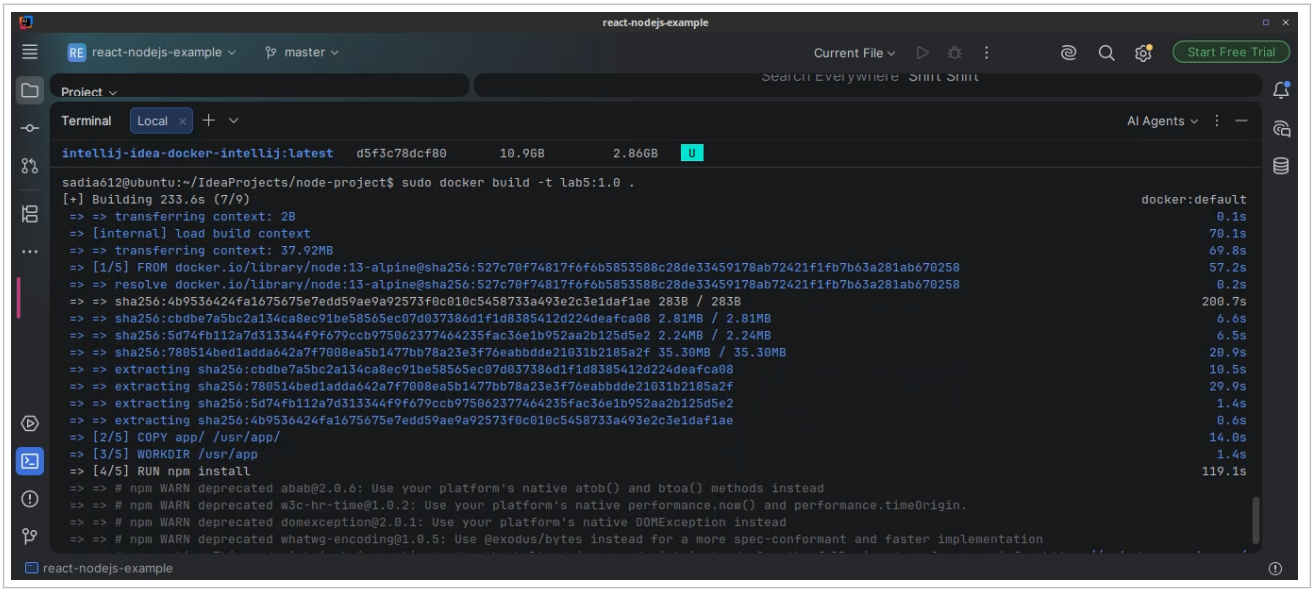


Figure 36: sudo docker build -t lab5:1.0 . completed and tagged the image. (docker_build.png)

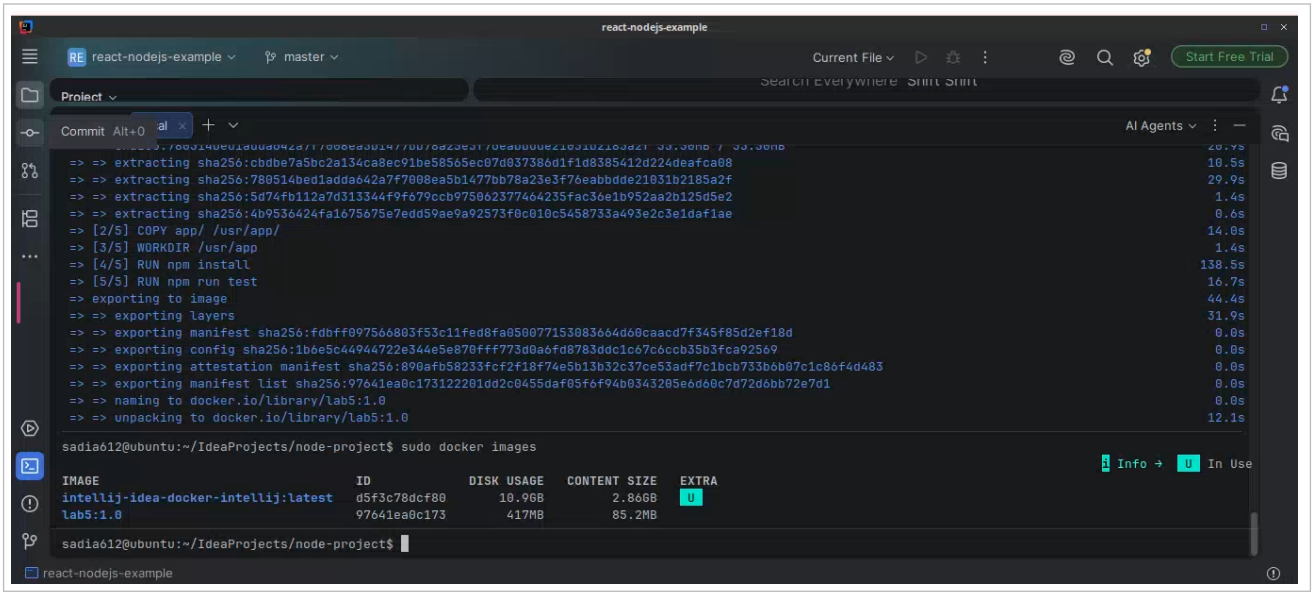


Figure 37: Image list after the build showing repository lab5 with tag 1.0. (docker_images_after.png)

Exam Evaluation Questions (practice answers)

1. Maven built java-maven-app-1.1.0-SNAPSHOT.jar. Which values in pom.xml determine this name, and what does the package phase do?

The JAR name is built from the **artifactId** (java-maven-app) and the **version** (1.1.0-SNAPSHOT) in pom.xml, in the form artifactId-version.jar. The **package** phase runs the earlier lifecycle phases first (validate, compile and test) and then packages the compiled classes and resources into a JAR in the target directory. With the Spring Boot plugin repackage goal, that JAR becomes an executable JAR.

2. Why can ./gradlew build work without a separate system-wide Gradle installation? Which JDK settings would you check if this sample reports an unsupported Java version?

The Gradle Wrapper (gradlew) reads gradle/wrapper/gradle-wrapper.properties and downloads the Gradle version the project needs on its first run, so no system-wide Gradle is required. For an unsupported Java version error (for example class file major version 65 with Gradle 7.x), check the **Project SDK**, the **language level**, the **Gradle JVM** in Settings, the **JAVA_HOME** variable, and confirm the JVM with ./gradlew --version. This sample needs JDK 16, not JDK 21.

3. Why does EADDRINUSE occur on port 3000 in this environment, and how does changing the application listener to port 4000 let the container browser open it?

The desktop (Webtop) inside the container already listens on port 3000, so the Node.js application cannot bind to the same port and fails with EADDRINUSE. Port 4000 is free, so the application starts there. The browser also runs inside the same container, so http://localhost:4000 reaches the application directly.